EOPS — Presentation Script for Four Speakers

ICSIE 2026 · 8 minutes of presentation + 2 minutes of questions · two slides per speaker

Speaker	Slides	Clock	Their job in one line
Sanjay P C	1 Title 2 Introduction	0:00 – 2:00	Open the talk and make the problem feel real
Sam Benadick J	3 Literature review 4 Methodology	2:00 – 4:20	Show the gap in prior work, then our architecture
Praveenkumar S	5 Experimental analysis 6 Results & discussion	4:20 – 6:40	How we measure it, and what is measured so far
Nibuna A	7 Conclusion 8 References	6:40 – 8:00	Land the contribution and invite questions

Rules for everyone

- Speak **slower than feels natural**. Pause wherever you see a grey cue.
- Finish with your handover sentence — that is how the next speaker knows to start.
- If the chair signals time, jump straight to the last paragraph of your part.
- Do not read the slide aloud. The slide is the evidence; you are the explanation.
- Four numbers you may state: **52.5 hours**, **0.766**, **6% versus 49%**, **11.34%**. Never guess any other number.

The same script is saved inside the .pptx as speaker notes, so it appears in PowerPoint's Presenter View during the talk.

SANJAY P C

Speaker 1 of 4 · slides 1 – 2 · 0:00 – 2:00

Slide 1 — Title · 0:00 – 0:45

Good morning, respected chairpersons and everyone present.

I am Sanjay P C. With me are Sam Benadick, Praveenkumar and Nibuna — final year students of Artificial Intelligence and Data Science at Sri Eshwar College of Engineering. Our work is guided by Ms D. Hemamalini.

Our paper is titled **“EOPS: A Multi-Agent AI Engineering Operations Platform for Automated Incident Investigation and Root Cause Analysis”**.

In the next eight minutes we will cover four things: the problem we are solving, the existing work and the gap in it, our proposed architecture and how we will evaluate it, and what we have already built. I will begin with the problem.

Slide 2 — Introduction · 0:45 – 2:00

When a software product breaks in production, the hardest part is not fixing it. The hardest part is **finding** what broke.

[point to the dark box]

A study of one full year of production incidents at Microsoft reported a median time to mitigate of **fifty-two and a half hours**. Most of that time is spent searching, not repairing.

The reason is that evidence is scattered. A single investigation touches six disconnected systems: the source code, the deployment history, the database, the application logs, the team’s runbooks, and old chat messages. The engineer opens all of them, holds partial clues in memory, and connects them by hand. It is slow, and the quality of the answer depends on which engineer happens to be on call that day.

Today’s AI assistants do not solve this. They generate code, or they answer questions. They do not run an investigation.

[point to the Objective box]

So our objective is a supervisor-driven multi-agent platform. Several specialised agents investigate the team’s own tools in parallel and return a root cause supported by evidence — with a human approving before anything is changed.

I will now hand over to Sam, who will present the related work.

SAM BENADICK J

Speaker 2 of 4 · slides 3 – 4 · 2:00 – 4:20

Slide 3 — Literature review · 2:00 – 3:10

Thank you, Sanjay.

[point to the top three rows]

We studied the literature in two groups. The first group is the foundation. Retrieval-Augmented Generation grounds a model's answer in retrieved documents, but it does not investigate anything. ReAct lets one agent reason and act using tools, but on a single task. AutoGen coordinates several agents in conversation, but the conversation is unbounded and nothing forces an agent to prove its claims.

[point to the lower rows]

The second group is our own domain. Ahmed and colleagues, at ICSE 2023, fine-tuned a model to suggest a root cause from the incident title and summary — a single shot, with no evidence gathering. RCACopilot, at EuroSys 2024, is the closest work to ours: it collects diagnostics through handlers and predicts a root-cause category with a **Micro-F1 of 0.766**. But it is provider-side, inside one company's own telemetry, and every alert type needs its own hand-written handler.

The most useful finding for us came from Roy and colleagues at FSE 2024. A tool-using agent was not much more accurate than the baselines — but its hallucination rate was **six percent, against forty-nine percent** for retrieval-based methods. Grounding, not model size, is what buys trust.

[point to the gap bar]

So the gap is clear. No system coordinates several specialised agents over a customer's own tools, sharing one investigation state in which every finding must cite the tool call that produced it.

Slide 4 — Methodology · 3:10 – 4:20

This is our proposed architecture.

[point to the top row]

The engineer describes the problem in a chat bar. A **Supervisor** agent reads it and plans the investigation. Importantly, the Supervisor may assign work only to desks that actually have a connected tool — it cannot invent an investigation it has no means to perform.

[point to the six boxes]

We then have one agent per system area — code, frontend and hosting, backend services, database, observability and knowledge. We call them desks. They run concurrently, each holding read-only tools for its own area, and a desk may ask one colleague a precise question when it needs information outside its area.

[point to the shared state bar]

Every desk writes into one shared investigation state, and every finding carries the identifier of the tool call that produced it. This is the core of our design: a claim without evidence is rejected by the system itself.

[point to the bottom row]

The RCA agent then reads only that collected evidence and produces the root cause, the exact failure point and a confidence value. Finally, the engineer approves before anything is written back to the company’s systems.

[point to the five numbered phases]

These are the five phases of the methodology. Each desk runs a budgeted reason-and-act loop — at most eight tool calls and three hundred seconds — and when a budget is reached the system forces the desk to report what it has, so an investigation can never hang.

Praveenkumar will now explain how we evaluate this.

PRAVEENKUMAR S

Speaker 3 of 4 · slides 5 – 6 · 4:20 – 6:40

Slide 5 — Experimental analysis · 4:20 – 5:25

Thank you, Sam. Our evaluation is designed as four experiments.

[point to each card in turn]

Experiment one tests orchestration: does the Supervisor coordinate the desks correctly and keep one consistent state? We measure orchestration latency, state consistency and the agent completion rate.

Experiment two tests the Engineering Memory Layer, our retrieval component. We measure Mean Reciprocal Rank, Recall at K, Precision at K and response latency, using real runbooks and postmortems.

Experiment three tests reasoning quality. Human evaluators compare our generated reports against expert analyses of the same incident, judging evidence use, completeness, explanation quality and confidence calibration.

Experiment four tests the complete workflow on injected faults — API latency, a slow query, a failed deployment, broken content — measuring time to root cause, evidence completeness, usability, and what we call the unsupported-claim rate.

[point to the dark bar]

We compare against three baselines: a single agent with the same tools, a one-shot prompt containing all the context, and our own architecture without compaction. We use the public **OpenRCA** benchmark with its three hundred and thirty-five real failures, **AIOpsLab** fault scenarios, and faults we inject into a live stack. The ablations isolate the architecture itself, rather than the model behind it.

PRAVEENKUMAR S · continued

Slide 6 — Results and discussion · 5:25 – 6:40

I will be precise about what we have measured and what we have not.

[point to the chart]

This is a measurement from our implementation. For one desk making eight tool calls, the earlier design resent two hundred and eighty-eight thousand characters of context across the task. With our compaction mechanism — where older tool results collapse to their opening but keep their citation identifier — it is one hundred and seventy-five thousand. About **thirty-nine percent less input**, with no loss of traceability.

[point to the three cards]

Second, context never grows with the length of the investigation, because each desk sees only its own tools and the RCA agent reads one-line evidence rather than raw logs. Third, because the desks run in parallel, total time follows the slowest desk, not the sum of all of them.

[point to the grey bar]

On implementation: nine agent roles, three live connectors, two hundred and twenty-nine automated tests, and a pilot investigation that ran end to end in about seven point seven minutes.

[point to the red line]

I want to state the limitation honestly. This is a position paper with a working prototype. That pilot timing is an engineering observation, **not** an evaluation result. The large-scale empirical study is our next phase, and its protocol is the one on the previous slide.

Nibuna will present the conclusion.

NIBUNA A

Speaker 4 of 4 · slides 7 – 8 · 6:40 – 8:00

Slide 7 — Conclusion · 6:40 – 7:40

Thank you, Praveenkumar. Four conclusions.

First, our contribution is a coordination protocol, not a bigger model. A supervisor plans, specialised desks investigate the team's own tools in parallel, and one shared state carries every finding together with its citation.

Second, trust comes from the harness, not from the model. Uncited findings are rejected. Investigating desks are read-only by construction. Every loop is bounded by a budget. And confidence is capped by code — for example, a conclusion supported by only one kind of source can never be reported above zero point six, no matter how certain the model sounds.

Third, decision support is the right target today. Published results show that even the best agents resolve only eleven to fifteen percent of real scenarios on their own. A system that acts on that would be dangerous. A system that investigates and hands the engineer cited evidence is useful right now.

Fourth, our future work: measurement on OpenRCA and on injected faults, graph-augmented retrieval across code, services and documents, and learning from which recommendations engineers accept and which they reject.

Slide 8 — References and thanks · 7:40 – 8:00

These are our references. Works eight to twelve position EOPS against the current state of the art in LLM-based root cause analysis.

On behalf of our team and our guide Ms D. Hemamalini, thank you for your time and attention.

We would be happy to take your questions.

Question time — who answers what

Two minutes. Only one person answers each question; the others stay quiet unless invited.

Question is about…	Answers	Opening words
The problem, motivation, who the users are	**Sanjay**	“The teams we are targeting are …”
Related work, “have you seen X?”, novelty, architecture	**Sam**	“Yes — the closest work is …”
Evaluation, metrics, baselines, datasets, cost, latency	**Praveen**	“We measure that as …”
Contribution, limitations, future work, human approval	**Nibuna**	“Our contribution there is …”

Three sentences that save you

When the question is unclear: “May I confirm — are you asking about …?”

When you do not know: “We have not measured that yet; it is part of our evaluation plan.”

When the criticism is fair: “That is a good point; we will include it in the next version.”

The hardest question, and its answer

“You have no results. Why should we accept this?”

“The contribution is the architecture and a working implementation; the paper states that explicitly. Our evaluation protocol is defined against OpenRCA, AIOpsLab and controlled fault injection, with four metric families. We would rather report a measured eleven percent than an unmeasured claim.”

Before you walk in

- ■ Each speaker has read their page aloud with a timer, finishing ten seconds early
- ■ The three handovers are rehearsed — that is where teams lose time
- ■ One person holds the clicker and advances slides for everyone
- ■ Presenter View opened once, so the notes layout is familiar
- ■ Deck copied to a pen drive *and* emailed to yourself as a backup